

Spring Framework vs Spring Boot

1. Introduction

Spring Framework and Spring Boot are closely related, but they are not the same thing.

A simple way to understand them is:

Spring Framework provides the core features needed to build Java enterprise applications. Spring Boot makes it easier and faster to build Spring-based applications by reducing configuration and setup work.

Spring Boot is built on top of the Spring ecosystem and uses many Spring Framework features internally.

2. What is Spring Framework?

Spring Framework is an open-source Java framework used for developing enterprise applications.

It provides important features such as:

- Inversion of Control
- Dependency Injection
- Spring Beans
- Spring MVC
- Transaction Management
- JDBC integration
- ORM integration
- AOP
- Testing support
- Integration support

The main goal of Spring Framework is to make Java enterprise development easier through modular architecture and loose coupling.

Example:

```
@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;

    public EmployeeService(EmployeeRepository employeeRepository) {
```

```
        this.employeeRepository = employeeRepository;
    }
}
```

Spring manages the `EmployeeService` and injects `EmployeeRepository`.

3. What is Spring Boot?

Spring Boot is a framework built on top of Spring that simplifies the creation of Spring applications.

It reduces the amount of manual configuration required.

Spring Boot provides:

- Auto Configuration
- Starter Dependencies
- Embedded Servers
- Externalized Configuration
- Production-ready Actuator features
- Simplified project setup
- Faster application development

Example:

```
@SpringBootApplication
public class EmployeeApplication {

    public static void main(String[] args) {

        SpringApplication.run(EmployeeApplication.class, args);

    }
}
```

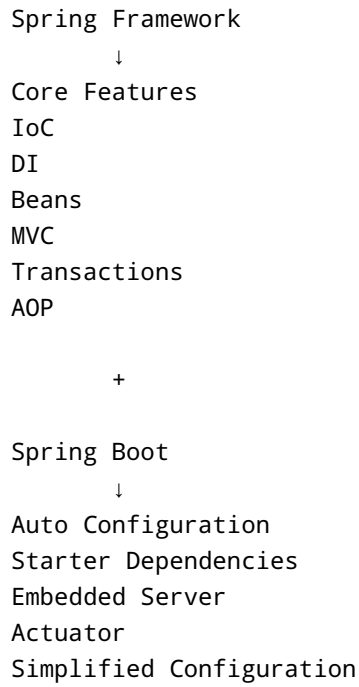
With this small amount of code, Spring Boot can start the Spring container and, for a web application, start an embedded server.

4. Relationship Between Spring and Spring Boot

Spring Boot does not replace Spring Framework.

Spring Boot uses Spring Framework internally.

Conceptually:



So:

Spring Boot = Spring Framework + Simplified Setup + Auto Configuration

5. Why Was Spring Boot Introduced?

Traditional Spring applications could require significant configuration.

Developers often had to configure:

- Spring beans
- DispatcherServlet
- View Resolver
- DataSource
- Transaction Manager
- Hibernate
- JDBC
- Web server

- Dependency versions

This could make application setup time-consuming.

Spring Boot was created to reduce this setup work.

6. Traditional Spring Configuration

A traditional Spring application may require XML configuration.

Example:

```
<bean id="employeeService"
      class="com.example.EmployeeService"/>
```

Component scanning might also be configured:

```
<context:component-scan
    base-package="com.example"/>
```

MVC setup may require additional configuration.

Developers may also configure the servlet container manually.

7. Spring Boot Configuration

Spring Boot reduces much of this configuration.

Example:

```
@SpringBootApplication
public class Application {

    public static void main(String[] args) {

        SpringApplication.run(Application.class, args);

    }
}
```

The application can then use:

```
@RestController  
@Service  
@Repository  
@Entity
```

Spring Boot automatically configures many required components based on the dependencies present in the application.

8. Major Difference: Configuration

Spring Framework

May require more manual configuration.

Example:

```
@Configuration  
@ComponentScan("com.example")  
@EnableWebMvc  
public class AppConfig {  
}
```

You may also need to configure:

- DataSource
- EntityManager
- TransactionManager
- Web server

Spring Boot

Uses Auto Configuration.

Example:

```
@SpringBootApplication  
public class Application {  
}
```

Spring Boot detects dependencies and configures many components automatically.

9. What is Auto Configuration?

Auto Configuration is one of the most important features of Spring Boot.

Spring Boot checks:

- Which dependencies are available
- Which classes exist
- Which beans already exist
- Which properties are configured

Based on this information, it automatically creates suitable configuration.

Example:

If the project contains:

```
spring-boot-starter-web
```

Spring Boot can configure:

- Spring MVC
- DispatcherServlet
- Jackson
- Embedded Tomcat
- Error handling infrastructure

10. Spring Boot Starter Dependencies

Traditional Spring applications may require multiple individual dependencies.

For a web application, developers might manually select:

- spring-web
- spring-webmvc
- jackson
- validation libraries
- servlet dependencies
- server libraries

Spring Boot groups compatible dependencies into starters.

Example:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

This gives a convenient set of dependencies required for web development.

11. Common Spring Boot Starters

Examples:

```
spring-boot-starter-web
spring-boot-starter-data-jpa
spring-boot-starter-security
spring-boot-starter-validation
spring-boot-starter-test
spring-boot-starter-actuator
```

This simplifies dependency management.

12. Embedded Server

In traditional Spring web applications, developers commonly deployed WAR files to an external application server.

Example:

```
Application WAR
  ↓
Tomcat Server
  ↓
Deploy Application
```

Spring Boot can embed a server directly in the application.

Default server for many web applications:

Tomcat

Application flow:

```
Spring Boot JAR
↓
Embedded Tomcat
↓
Application Starts
```

Run using:

```
java -jar employee-service.jar
```

This is one of the major advantages of Spring Boot.

13. Spring vs Spring Boot Deployment

Traditional Spring

Common process:

```
Develop Application
↓
Build WAR File
↓
Install Tomcat
↓
Configure Tomcat
↓
Deploy WAR
↓
Start Server
```

Spring Boot

Common process:

Develop Application



Build JAR



Run JAR

Command:

```
java -jar application.jar
```

14. Spring MVC in Spring Framework

Spring Framework provides Spring MVC.

Typical architecture:

```
Client
  ↓
DispatcherServlet
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

But developers may need additional MVC configuration.

15. Spring MVC in Spring Boot

Spring Boot uses Spring MVC internally when the appropriate starter is included.

Example:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Then:

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

    @GetMapping
    public String getEmployees() {

        return "Employees";
    }
}
```

Spring Boot automatically configures the web infrastructure.

16. Database Configuration

Traditional Spring

You may manually configure:

```
DataSource
EntityManagerFactory
TransactionManager
Hibernate Properties
```

Example:

```
@Bean
public DataSource dataSource() {

    // manual configuration
}
```

Spring Boot

You can often configure the database using properties.

Example:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/employeedb
spring.datasource.username=postgres
spring.datasource.password=admin

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Spring Boot can automatically create the required infrastructure.

17. Application Configuration

Spring Boot commonly uses:

```
application.properties
```

or:

```
application.yml
```

Example:

```
server.port=8081
```

This changes the application server port.

Traditional Spring applications often required more explicit configuration.

18. Main Annotation in Spring Boot

The main Spring Boot annotation is:

```
@SpringBootApplication
```

This is essentially a convenience annotation combining:

```
@SpringBootApplication
@EnableAutoConfiguration
@ComponentScan
```

Conceptually:

```
@SpringBootApplication
  ↓
Configuration
  +
Auto Configuration
  +
Component Scanning
```

19. Component Scanning

Spring detects classes annotated with:

```
@Component
@Service
@Repository
@Controller
@RestController
```

Spring Boot automatically scans packages below the main application package.

Example:

```
com.example.employee
|
├─ EmployeeApplication.java
|
├─ controller
├─ service
└─ repository
```

If the main class is in:

```
com.example.employee
```

Spring Boot automatically scans the child packages.

20. Boilerplate Code

Spring

May require more boilerplate configuration.

Example:

```
Configure MVC  
Configure Beans  
Configure Database  
Configure Server  
Configure Dependencies
```

Spring Boot

Reduces boilerplate:

```
Add Starter  
Configure Properties  
Write Business Code  
Run Application
```

21. Development Speed

Spring Boot generally offers faster development because much of the setup is already handled.

Traditional Spring:

```
Setup  
Configuration  
Server Setup
```

```
Dependency Setup
Application Code
```

Spring Boot:

```
Starter Dependencies
Configuration Properties
Application Code
Run
```

22. Microservices

Spring Framework can be used to build microservices.

However, Spring Boot is usually preferred because microservices benefit from:

- Independent deployment
- Embedded servers
- Smaller application units
- Simple configuration
- Fast startup/setup
- REST API support
- Easy packaging

Typical flow:

```
Client
  ↓
API Gateway
  ↓
Employee Service
Order Service
Payment Service
Inventory Service
```

Each service can be an independent Spring Boot application.

23. Spring Boot Actuator

Spring Boot provides Actuator for production monitoring.

Example dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

It can expose endpoints for:

- Health
- Metrics
- Environment
- Beans
- Application information

Example:

```
/actuator/health
```

Spring Framework alone does not provide the same Spring Boot Actuator experience.

24. Exception Handling

Spring Framework provides mechanisms such as:

```
@ExceptionHandler
@ControllerAdvice
```

Spring Boot builds on them and also provides default error handling.

Example:

```
@RestControllerAdvice
public class GlobalExceptionHandler {
}
```

25. Testing

Both Spring and Spring Boot support testing.

Spring Boot provides convenient testing support through:

```
spring-boot-starter-test
```

Common annotations:

```
@SpringBootTest  
@WebMvcTest  
@DataJpaTest
```

This makes testing Spring Boot applications easier.

26. Dependency Management

Spring applications require compatible dependency versions.

In a traditional setup, developers may need to manage them manually.

Spring Boot simplifies this through managed dependency versions.

Example:

```
<parent>  
  
  <groupId>org.springframework.boot</groupId>  
  
  <artifactId>  
    spring-boot-starter-parent  
  </artifactId>  
  
</parent>
```

This helps maintain compatible library versions.

27. Project Creation

Traditional Spring projects may require manual Maven/Gradle setup.

Spring Boot projects can be quickly created using:

Spring Initializr

Typical selections:

Spring Web
Spring Data JPA
PostgreSQL Driver
Lombok
Validation
Spring Boot DevTools

This greatly speeds up project setup.

28. Spring vs Spring Boot Architecture

Spring Framework

```
Application
  ↓
Manual / Explicit Configuration
  ↓
Spring Container
  ↓
Beans
  ↓
Business Logic
```

Spring Boot

```
Application
  ↓
@SpringBootApplication
  ↓
```

Auto Configuration

↓

Spring Container

↓

Beans

↓

Embedded Server

↓

Business Logic

29. Spring Framework Example

```
@Configuration
@ComponentScan("com.example")
public class AppConfig {
}
```

Application startup:

```
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);
```

Developers explicitly create the application context.

30. Spring Boot Example

```
@SpringBootApplication
public class EmployeeApplication {

    public static void main(String[] args) {

        SpringApplication.run(
            EmployeeApplication.class,
            args
        );

    }
}
```

Spring Boot handles much of the startup process.

31. Real-Time Application Example

Suppose we build an Employee Management REST API.

Architecture:

```
Postman / Angular
    ↓
EmployeeController
    ↓
EmployeeService
    ↓
EmployeeRepository
    ↓
PostgreSQL
```

With Spring Framework, we may need to configure more infrastructure manually.

With Spring Boot, we can add:

```
Spring Web
Spring Data JPA
PostgreSQL Driver
Validation
```

and quickly start writing business logic.

32. Detailed Comparison Table

Feature	Spring Framework	Spring Boot
Type	Core Java framework	Framework built on Spring
Primary Goal	Enterprise application development	Simplify Spring application development
Configuration	More manual configuration	Auto Configuration

Feature	Spring Framework	Spring Boot
Dependencies	Can require manual dependency setup	Starter Dependencies
Web Server	External server common traditionally	Embedded server supported
Deployment	WAR commonly used	Executable JAR common
Setup Time	Higher	Lower
Boilerplate	More	Less
Dependency Versions	More developer management	Boot manages many versions
Application Properties	Supported	Strong convention-based configuration
Microservices	Possible	Very commonly used
Production Monitoring	Requires separate setup	Actuator available
Startup	More explicit setup	Simplified
Learning	Requires Spring fundamentals	Easier setup, but Spring concepts still important
Development Speed	Moderate	Faster
Testing	Spring Test support	Boot test starters and slices
REST APIs	Supported through Spring MVC	Very easy with starters
Database Integration	Spring JDBC / ORM	Easy auto configuration with Data JPA
Server Configuration	Usually more manual	Easy through properties
Packaging	WAR common historically	JAR common

33. Advantages of Spring Framework

Spring Framework provides:

- Strong IoC Container
- Dependency Injection
- Loose coupling
- Modular architecture

- Transaction management
- Spring MVC
- JDBC integration
- ORM integration
- AOP
- Enterprise integration
- Testing support

Spring Framework gives developers very fine-grained control.

34. Advantages of Spring Boot

Spring Boot provides:

- Faster development
 - Auto Configuration
 - Starter Dependencies
 - Embedded server
 - Easy project setup
 - Simple deployment
 - Reduced boilerplate
 - Actuator
 - Configuration through properties
 - Microservice-friendly architecture
-

35. Disadvantages of Spring Framework

Possible challenges include:

- More configuration
 - Longer setup time
 - Dependency management complexity
 - External server setup in traditional applications
 - Steeper initial learning curve
-

36. Disadvantages of Spring Boot

Spring Boot also has some challenges:

- Auto Configuration can hide internal behavior from beginners.
- Applications may include more dependencies than a minimal custom setup.

- Startup and memory usage may be higher than very lightweight frameworks.
 - Developers still need to understand Spring fundamentals.
 - Incorrect auto configuration assumptions can sometimes make debugging harder.
-

37. Is Spring Boot Better Than Spring Framework?

This is not exactly the correct comparison.

Spring Boot depends on Spring Framework.

You cannot think of Spring Boot as replacing Spring.

A better way to understand it is:

```
Spring Framework
  ↓
Foundation

Spring Boot
  ↓
Simplified Way to Use Spring
```

Spring Boot makes it easier to use Spring Framework.

38. Can We Learn Spring Boot Without Spring?

Technically, you can start building Spring Boot applications directly.

However, to understand Spring Boot properly, you should know:

- IoC
- Dependency Injection
- Beans
- Bean lifecycle
- Bean scopes
- ApplicationContext
- Component scanning
- Spring MVC
- Constructor injection

Because Spring Boot internally uses all these Spring Framework concepts.

39. Recommended Learning Order

A good learning sequence is:

```
Core Java
↓
Spring Framework Basics
↓
IoC
↓
Dependency Injection
↓
Spring Beans
↓
ApplicationContext
↓
Spring MVC
↓
Spring Boot
↓
REST API
↓
Spring Data JPA
↓
Database
↓
Exception Handling
↓
Validation
↓
Security
↓
Microservices
```

40. When Should We Use Spring Framework?

Spring Framework directly can be useful when:

- You need complete control over configuration.
- You are working with an existing traditional Spring application.
- You need to understand or maintain legacy Spring applications.

- You want deeper control over framework configuration.

41. When Should We Use Spring Boot?

Spring Boot is preferred for most modern Spring applications such as:

- REST APIs
- Microservices
- Web applications
- Backend applications
- Cloud-native services
- Enterprise APIs
- Containerized applications

42. Simple Student Explanation

Students can remember it like this:

SPRING FRAMEWORK

Provides:

IoC

DI

Beans

MVC

AOP

Transactions

Database Integration

But:

Requires More Setup

SPRING BOOT

Uses Spring Framework

+

Auto Configuration

+

Starter Dependencies

+

Embedded Server

+

Actuator

Result:
Faster Development

43. Real-Time Analogy

Think about building a house.

Spring Framework

Spring gives you:

- Cement
- Bricks
- Steel
- Tools
- Construction methods

But you need to decide how everything is configured.

Spring Boot

Spring Boot gives you:

- Spring materials
- Preconfigured setup
- Standard structure
- Ready-to-use tools

So construction starts faster.

The underlying technology is still Spring.

44. Common Interview Question

What is the difference between Spring and Spring Boot?

A good interview answer:

Spring Framework is the core Java framework that provides features such as IoC, Dependency Injection, Spring MVC, transaction management, AOP, and data-access integration. Spring Boot is built on top of Spring and simplifies application development by providing auto configuration, starter dependencies, embedded servers, externalized configuration, and production-ready features. Spring Boot reduces manual configuration and makes it faster to build and deploy Spring applications.

45. Short Interview Answer

Spring provides the core framework features, while Spring Boot simplifies the configuration, development, and deployment of Spring applications.

46. One-Line Difference

Spring Framework = Core Features

Spring Boot = Spring Framework + Auto Configuration + Easy Setup

47. Final Summary

Spring Framework is the foundation.

It provides:

IoC
DI
Beans
MVC
AOP
Transactions
Data Access
Testing

Spring Boot builds on that foundation and adds:

Auto Configuration
Starter Dependencies

Embedded Server
Actuator
Easy Configuration
Simplified Deployment

Therefore:

Spring Framework
↓
Powerful Foundation

Spring Boot
↓
Simplified Development Experience

The most important point for students to remember is:

Spring Boot does not replace Spring Framework. Spring Boot uses Spring Framework and makes Spring application development faster, easier, and more production-friendly.